Exact Implementation Specification: Slices Perspective

Faithful implementation plan for "A Slices Perspective for Incremental Nonparametric Inference in High Dimensional State Spaces"

0. Purpose and fidelity rules

This document specifies how to implement the Slices Perspective method as described by Shienman, Levy-Or, Kaess, and Indelman. It is written as an implementation handoff for an engineer or AI coding agent. The goal is not to improve the method, but to reproduce the authors' construction as accurately as the paper specifies.

Fidelity rule

When the paper is explicit, implement exactly. When the paper is not explicit, expose the choice as a configurable module and mark it as an implementation assumption. This matters especially for the MMD kernel/estimator and for practical sampling from arbitrary factors.

Source anchor: The paper defines the method as using high-dimensional slices to approximate posterior distributions without additional intermediate reconstructions; see abstract and contributions, p. 1; methodology and estimator, pp. 4-5.

1. Inputs, outputs, and core objects

Symbol / object	Implementation meaning	Required data structure
G=(F, Θ, E)	Bipartite factor graph with factor nodes F, variable nodes Θ, and edges E.	FactorGraph object with factor list, variable list, adjacency maps.
f_i(Θ_i)	Unary or pairwise factor. The paper assumes factors connect to up to two variables.	Factor object with scope, logValue/evaluate, sampleIfPossible, measurement info.
D	All available data: prior b0, actions, and observations. In intermediate elimination D_j may include approximated f_new.	DataStore containing raw factors plus generated approximate factors.
O = ord(Θ)	Variable elimination ordering. Paper examples start from a variable linked to a prior.	Ordered variable list; for Plaza use affected-variable re-elimination with landmarks eliminated last.
N	Number of samples used at each sampling/approximation step.	Integer hyperparameter. Paper uses 150 samples per step in Plaza2 and 200 in Four Doors comparisons.
f_new(S_j | D_j)	New separator factor generated after eliminating ω_j.	ApproximateFactor object storing slice mixture and evaluators.
P(ω_j | S_j, D_j)	Bayes-net conditional generated during elimination.	ConditionalFactor object capable of evaluating conditional slices and sampling during backward pass.
MMD settings	Backward-pass early stopping heuristic settings.	N_M samples and threshold ϑ; paper uses N_M=100 and ϑ=1e-4 for Plaza2.

2. Exact forward-backward structure before approximation

The implementation must first mirror the paper's exact variable-elimination formulation. Do not begin with KDEs, flows, or generic particle filters. The Slices method is built on the following local exact objects.

$$\textit{Eq. S1: } P(\Theta \mid D) \propto \prod_i f_i(\Theta_i)$$

$$\textit{Eq. S2: } F_{j-1}(\omega_j) = \{ f \mid e(f, \omega_j) \in E_{j-1} \}$$

$$\textit{Eq. S3: } S_j = \text{variables connected to removed factors except } \omega_j$$

$$\textit{Eq. S4: } P_{joint}(\omega_j, S_j \mid D_j) = \eta_j^{-1} \prod_{f_i \in F_{j-1}(\omega_j)} f_i(\Theta_i)$$

$$\textit{Eq. S5: } P_{joint}(\omega_j, S_j \mid D_j) = P(\omega_j \mid S_j, D_j)\, f_{new}(S_j \mid D_j)$$

$$\textit{Eq. S6: } f_{new}(S_j \mid D_j) = \eta_j^{-1} \int_{\omega_j} \prod_{f_i \in F_{j-1}(\omega_j)} f_i(\Theta_i)\, d\omega_j$$

Source anchor: Equations correspond to paper Eq. (3)-(7) and Eq. (11)-(12): factor graph, forward elimination, separator, local joint, chain-rule factorization, and backward marginalization; see pp. 3-4.

3. Forward pass implementation

The forward pass transforms the factor graph into a Bayes net. Each elimination removes one variable, creates one conditional, and inserts one new factor over the separator.

```
Algorithm S1: ForwardPassUsingSlices(G, O, N)
Input: factor graph G0=(F0, 00, E0), elimination order O={ω1,...,ω|0|}, sample count N
Output: BayesNet conditionals C_j and reduced/generated factors

for j = 1,...,|0|:
    1. Identify current variable ω_j in G_{j-1}.
    2. Collect removed factors F_{j-1}(ω_j).
    3. Define separator S_j from all neighbors in removed edges except ω_j.
    4. Define P_joint(ω_j,S_j|D_j) as product of removed factors.
    5. Approximate f_new(S_j|D_j) by the slice estimator in Algorithm S2.
    6. Build conditional estimator P_hat(ω_j|S_j,D_j) by Algorithm S3.
    7. Add P_hat(ω_j|S_j,D_j) to Bayes net.
    8. Remove ω_j and its removed factors from G_{j-1}.
    9. Insert f_hat_new(S_j|D_j) as a new factor into G_j.
return all conditionals and generated factors
```

4. Slice estimator for f_new

The paper's estimator samples the eliminated variable ω_j from some factor f^∘ in F_{j-1}(ω_j). The implementation should keep the same structure: the sampled factor is not evaluated in the product because it serves as the sampling source; all other factors are evaluated on the sampled ω_j.

$$\textit{Eq. S7: } \omega_j^n \sim \text{sample from some factor } f^\circ \in F_{j-1}(\omega_j)$$

$$\textit{Eq. S8: } \hat{f}_{new}(S_j \mid D_j) = (\eta_j^{-1}/N) \sum_{n=1}^{N} \prod_{f_i \in F_{j-1}(\omega_j) \setminus \{f^\circ\}} f_i(\omega_j^n, \Theta_i \mid \{\omega_j\})$$

```
Algorithm S2: EstimateNewFactorBySlices(ω_j, S_j, removedFactors, N)
Input: eliminated variable ω_j, separator S_j, factors F_{j-1}(ω_j), sample count N
Output: ApproximateFactor f_hat_new(S_j|D_j)

1. Choose a sampling factor f^∘ from F_{j-1}(ω_j).
   Priority: unary factor if available; otherwise use the structural rule of Lemma 1.
2. Draw N samples {ω_j^n}_{n=1}^N from f^∘.
3. For a requested separator value s, evaluate:
      value(s) = (η_j^{-1}/N) * sum_n product over all removed factors except f^∘ evaluated at (ω_j^n, s).
4. Store the samples and the evaluable mixture-of-slices representation.
5. Return f_hat_new.
```

Source anchor: Paper Eq. (13) and explanation: samples of ω_j are generated from some factor f^∘ and each remaining factor is evaluated using those samples; p. 4.

5. Sampling when no unary factor exists: Lemma 1 implementation

The key implementation issue is finding samples for ω_j. The paper's Lemma 1 guarantees this under an elimination-order condition: each eliminated variable either has a unary factor, or is connected to a previously eliminated variable.

```
Algorithm S2a: SelectSamplingFactorByLemma1(ω_j, G_{j-1}, eliminationHistory)
if exists unary factor f(ω_j) in F_{j-1}(ω_j):
    return f(ω_j)
else:
    find previously eliminated ω_i such that f(ω_i,ω_j) was in original F
    locate the propagated/generated factor in F_{j-1}(ω_j) that contains samples involving ω_j
    return that factor/slice structure as the sampling source
```

```
if no such source exists:
    raise EliminationOrderError
```

Source anchor: Lemma 1 and proof by induction: samples can be drawn from a factor in F_{j-1}(ω_j) if the ordering condition holds; see p. 5 and appendix p. 7.

6. Worked implementation pattern for the paper example

Implement the example exactly as a test fixture. The graph subset is x1, x2, l1 with elimination order ω1=x1, ω2=l1. This fixture is essential because it tests the case where the second eliminated variable has no unary factor.

Eq. S9: after eliminating x1: f_hat_new(l1, x2 | D1) is added to the graph

Eq. S10: P_joint(l1, x2 | D2) = f_hat_new(l1, x2 | D1) f(l1, x2)

Eq. S11: P_joint(l1, x2 | D2) = P(l1 | x2, D2) f_new(x2 | D2), with S2={x2}

Eq. S12: D2 = { f_hat_new(l1, x2 | D1), z2 }, not the full raw data set

Eq. S13: f_tilde_new(x2 | D2) = (η^{-1}/N) ∑_n f(x1^n,x2) ∫ f(x1^n,l1) f(l1,x2) dl1

Eq. S14: f_hat_new(x2 | D2) = (η^{-1}/N^2) ∑_{n1=1}^N f(x1^{n1},x2) ∑_{n2=1}^N f(l1^{n1,n2},x2)

Source anchor: Example around paper Eqs. (14)-(16): D2 contains the approximate factor from the previous elimination; l1 samples are generated from the unary structure f(x1^n,l1); p. 5.

7. Conditional estimator implementation

Eq. S15: P_hat(ω_j | S_j, D_j) = [∏_{f_i∈F_{j-1}(ω_j)} f_i(Θ_i)] / [(1/N)∑_{n=1}^N ∏_{f_i∈F_{j-1}(ω_j)\{f^∘}} f_i(ω_j^n, Θ_i\{ω_j})]

The normalizing term η_j^{-1} cancels. In implementation, evaluate numerator directly from current factors and denominator through f_hat_new's slice mixture at the current separator value.

```
Algorithm S3: BuildConditionalEstimator(ω_j, S_j, removedFactors, f_hat_new)
Input: removed factors and the approximate separator factor from Algorithm S2
Output: evaluator/sampler for P_hat(ω_j | S_j, D_j)

def conditional_value(ω, s):
    numerator = product of all removed factors at (ω, s)
    denominator = f_hat_new.evaluate(s), with η ignored/cancelled consistently
    return numerator / denominator

Store conditional_value as a Bayes-net conditional.
```

Source anchor: Paper Eq. (17) and note that η_j^{-1} cancels; p. 5.

8. Backward pass and posterior sampling

Eq. S16: P_hat(ω_j | D) = (1/N) ∑_{n=1}^N P_hat(ω_j | S_j^n, D_j), S_j^n ~ P_hat(S_j | D)

```
Algorithm S4: BackwardPassAndMarginals(BayesNet, reverse(O), N)
Input: Bayes-net conditionals from forward pass, reverse elimination order
Output: marginal distributions and optional joint posterior samples

1. Start from the last eliminated variable; its separator is empty.
2. For each variable ω_j in reverse elimination order:
      a. Draw N separator samples S_j^n from the already available marginal P_hat(S_j|D).
      b. Build marginal mixture P_hat(ω_j|D) = (1/N) sum_n P_hat(ω_j | S_j^n, D_j).
      c. Cache this marginal for later variables and for incremental early stopping.
```

```
3. For joint posterior samples: sample last variable first, then recursively sample each variable from
P_hat(ω_j|S_j,D_j).
```

Source anchor: Paper Eq. (18) and joint posterior sampling description: marginals are mixtures of high-dimensional slices and joint samples follow reverse elimination order; p. 5.

9. Incremental inference and early stopping

The paper proposes re-elimination and an MMD-based early stopping heuristic for the backward pass. Implement this as a cache-aware update procedure.

```
Algorithm S5: IncrementalUpdateWithMMDEarlyStop(newFactors, previousGraph, previousCaches)
Input: new measurements/factors, old factor graph, cached generated factors and marginals
Output: updated graph, updated factors, updated marginals

1. Add new factors to the factor graph.
2. Determine affected variables/subgraph.
3. Re-eliminate only affected variables using the selected re-elimination order.
4. Run backward pass from the affected region.
5. For each newly computed marginal P_new(ω|D), compare to cached P_old(ω|D) using MMD.
6. If MMD(P_new, P_old) < ϑ, stop the backward pass.
7. Otherwise, cache P_new and continue to the next variable in reverse order.
```

Item	Paper-specified setting / behavior	Implementation handling
MMD metric	Used to compare new and cached marginal distributions in backward pass.	Implement a sample-based MMD function; expose kernel and bandwidth as configurable because the paper does not specify them.
Stopping rule	Stop backward pass if MMD is below a user-defined threshold.	if MMD < ϑ: stop and reuse cached downstream marginals.
Hyperparameters	Two hyperparameters: number of samples for MMD and threshold.	Use N_M and ϑ in configuration.
Plaza2 values	N_M = 100 and ϑ = 1e-4.	Default these values for the Plaza2 reproduction profile.
Re-elimination order	Affected-variable order chosen so landmark variables are eliminated last.	For Plaza2 reproduction, enforce landmarks-last order.
Samples per step	Slices configured with 150 samples in each step for Plaza2.	Set N=150 in Plaza2 reproduction profile; use N=200 in Four Doors comparison profile when reproducing Fig. 4/5 context.

Source anchor: Incremental inference, re-elimination, MMD early stopping, N_M=100, ϑ=1e-4, landmarks-last, and 150 samples per step are given in pp. 5-7.

10. Required software architecture

Module	Responsibilities
FactorGraph	Add/remove factors and variables, adjacency queries, affected-subgraph extraction.
Factor	Represent unary/pairwise factors; evaluate log-density/value; sample if possible; hold measurement model.
ApproximateFactor	Represent f_hat_new as a mixture of slices; evaluate at separator values; generate samples when used by Lemma 1.
ConditionalFactor	Evaluate P_hat(ω_j|S_j,D_j); sample given separator realizations during backward pass.
EliminationEngine	Forward pass, factor removal, separator discovery, new factor insertion, Bayes-net construction.
SamplingEngine	Factor-based sampling and nested sample generation per Lemma 1.
MMDStopper	Compute sample-based MMD between cached and new marginal samples; implement early stop.
IncrementalManager	Maintain caches, affected subgraph, re-elimination order, and reuse of old marginals/factors.
ExperimentProfiles	Four Doors and Plaza2 settings, sample counts, order conventions, metrics.

11. Minimum tests for author-faithful reproduction

Test	Pass condition
Forward exactness on toy graph	The Bayes net conditionals follow the example factorization P(x1|x2,l1) P(l1|x2) P(x2|x3,l2) P(l2|x3) P(x3).
No-unary test	After eliminating x1, l1 has no original unary factor but samples are still generated using f_hat_new(l1,x2|D1).

Conditional estimator cancellation	η_j^{-1} is not required to evaluate Eq. S15; numerator/denominator ratio is stable for supported samples.
Backward marginal mixture	P_hat(ω_j|D) is represented as a mixture of conditional slices over separator samples.
MMD early stop	With MMD below ϑ, backward pass terminates and old cached marginals are reused beyond that point.
Plaza2 reproduction defaults	N=150, N_M=100, ϑ=1e-4, landmarks-last re-elimination order.

12. Source references

- Shienman, M.; Levy-Or, O.; Kaess, M.; Indelman, V. “A Slices Perspective for Incremental Nonparametric Inference in High Dimensional State Spaces,” IROS 2024.
- Key source anchors used: paper Eq. (3)-(7) for factor graph and forward/backward elimination; Eq. (8)-(10) for generic slices; Eq. (11)-(13) for forward pass estimator; Lemma 1 and appendix for sample generation; Eq. (14)-(18) for the example, conditional estimator, and backward pass; Section IV-B/V-B for MMD early stopping and Plaza2 hyperparameters.

13. Detailed implementation pipeline for an AI coding agent

The following pipeline is the recommended coding order. It prevents the most common mistake: implementing a generic particle filter or KDE reconstruction instead of the paper's mixture-of-slices estimator.

```
Recommended build order:
1. Implement Factor and FactorGraph without any Slices logic.
2. Implement exact elimination bookkeeping: removed factors, separator S_j, reduced graph G_j.
3. Implement ApproximateFactor as a lazy evaluable mixture of slices.
4. Implement factor-based sampling for unary factors.
5. Implement Lemma-1 structural sampling for variables with no unary factor.
6. Implement f_hat_new evaluator using Eq. S8.
7. Implement conditional estimator using Eq. S15.
8. Implement backward marginal mixture using Eq. S16.
9. Add incremental caches and MMD early stopping.
10. Build only the demonstrator/visualization after the math engine passes tests.
```

Implementation phase	Do this	Do not do this
Factor graph core	Represent factors and adjacency exactly; keep variable ordering explicit.	Do not collapse factors to Gaussian approximations.
Forward pass	Create f_hat_new by evaluating high-dimensional slices at sampled ω_j values.	Do not reconstruct f_new with KDE unless implementing a comparison baseline.
No unary factor	Use Lemma 1 to locate a sampleable propagated factor/slice structure.	Do not break the graph into an ancestral chain as the Slices paper explicitly avoids this.
Backward pass	Use mixture of conditional slices over separator samples.	Do not compute only a mean or MAP value.
Incremental mode	Re-eliminate affected variables and stop backward pass when MMD is below threshold.	Do not recompute the complete batch pass at every step unless running a baseline.

Source anchor: The paper distinguishes itself by avoiding intermediate reconstructions and by generating samples without graph decomposition, even without unary factors; see pp. 1, 4-5.

14. Data structures in more detail

Class	Fields	Critical methods
Variable	name, dimension, type {pose, landmark, observation}, manifold metadata.	equals/hash, displayName, supportDomain.
Factor	id, scope variables Θ_i, measurement z_i, noise model, factor type.	evaluate(values), logEvaluate(values), canSample(targetVar,givenValues), sample(targetVar,givenValues,N).
ApproximateFactor	scope=S_j, eliminatedVar=ω_j, sourceSamples, sourceFactor id, remainingFactors, normalizer flag.	evaluate(separatorValues), sample(targetVar,N), describeMixture().
ConditionalFactor	frontal=ω_j, separator=S_j, numeratorFactors, denominator ApproximateFactor.	evaluate(ω_j,separator), sampleGiven(separator,N), denominatorDiagnostics.
BayesNet	list of ConditionalFactor in elimination order.	appendConditional, sampleJointReverseOrder, evaluateJointProduct.
MarginalCache	variable id, sample set, mixture representation, timestamp, source step.	compareMMD(newMarginal), update, retrieveSeparatorSamples.
IncrementalState	current graph, generated factors, conditionals, marginal cache, ordering.	applyNewFactors, findAffectedVariables, reEliminate, earlyStopBackward.

15. Exact handling of constants and normalization

The paper carries η_j^{-1} as a normalizing term, but notes that it cancels in the conditional estimator and does not need explicit calculation for intermediate new factors used only for sampling separators. The implementation should therefore support two modes: unnormalized factor evaluation for propagation and normalized density evaluation only when required for metrics or sampling diagnostics.

Eq. S17: conditional ratio = full product / estimated marginal factor; η_j^{-1} cancels

Use case	Need η_j?	Implementation rule
Forward generated factor used as next factor	No, if only relative weights/sampling structure are needed.	Store scale as unknown; keep evaluations consistent.
Conditional estimator Eq. S15	No.	Use numerator and denominator with same

		convention.
Plotting normalized marginal	Yes or approximate.	Normalize numerically over displayed support or samples.
Computing MMD	No explicit density normalization required if using samples.	Draw samples from marginal mixture and compare sample sets.

Source anchor: The paper states that η_j^{-1} cancels in the conditional estimator and need not be explicitly calculated for intermediate new factors; p. 5.

16. MMD early stopping implementation details

The paper specifies MMD as the backward-pass stopping metric, with sample count and threshold as hyperparameters. It does not specify the kernel or biased/unbiased MMD estimator. Therefore, expose these choices, while keeping the paper's high-level stopping rule exact.

```
Function shouldStopByMMD(oldMarginal, newMarginal, N_M, theta):
    X = oldMarginal.drawSamples(N_M)
    Y = newMarginal.drawSamples(N_M)
    d = MMD(X, Y; kernel=config.kernel, bandwidth=config.bandwidth)
    return d < theta, d

Default reproduction profile for Plaza2:
    N = 150
    N_M = 100
    theta = 1e-4
    re-elimination order: landmarks last
```

Source limitation

The paper provides the MMD concept, thresholding rule, N_M and threshold values for Plaza2, but not the exact MMD kernel/bandwidth. A faithful implementation must therefore make the kernel explicit as an assumption in the code configuration.